

深圳大学实验报告

课程名称： 算法设计与分析

实验名称： 实验三 回溯法—地图填色问题

学院： 计算机与软件学院 专业： 软件工程

报告人： 黄瑜 学号： 2024150080 班级： 软件工程 01

同组人： 黄家成 2024150286

指导教师： 吴定明

实验时间： 2026 年 5 月 17 日

实验报告提交时间： 2026 年 5 月 17 日

一. 实验目的

- (1) 掌握回溯法算法设计思想。
- (2) 掌握地图填色问题的回溯法解法。

二. 实验内容与结果

四色问题：“任何一张地图只用四种颜色就能使具有共同边界的国家着上不同的颜色。”
也就是说在不引起混淆的情况下一张地图只需四种颜色来标记就行。

问题描述：

我们可以将地图转换为平面图，每个地区变成一个节点，相邻地区用边连接，我们要为这个图形的顶点着色，并且两个顶点通过边连接时必须具有不同的颜色。

附件是给出的地图数据，请针对三个地图数据尝试分别使用 5 个（le450_5a），15 个（le450_15b），25 个（le450_25a）颜色为地图着色。

1. 基础回溯法与小规模数据测试（实验要求 1）

(1) 算法设计思想

回溯法是一种“试探性”的搜索算法。在地图填色问题中，我们从第 1 个顶点开始，逐个为其分配颜色。在为第 v 个顶点分配颜色时，遍历所有可选颜色（1 到 m ）。如果当前颜色与周围已着色的邻居节点发生冲突，则尝试下一种颜色；如果所有颜色都冲突，则说明前方的着色方案走入了“死胡同”，算法将“回溯”到上一个顶点，更改其颜色后继续试探。

(2) 核心伪代码

```
1 函数 Backtrack(v):
2     如果 v > 顶点总数:
3         返回 True // 所有顶点着色完毕，找到解
4
5     对于颜色 c 从 1 到 m:
6         如果 is_safe(v, c) == True: // 检查是否与邻居冲突
7             colors[v] = c // 做出选择
8             如果 Backtrack(v + 1) == True:
9                 返回 True // 递归进入下一层
10            colors[v] = 0 // 撤销选择，回溯
11
12    返回 False // 所有颜色均不可行，触发回溯
```

3) 测试结果

为了验证算法的正确性，我们构造了一个包含 4 个顶点和一条对角线的简单图（即一个带有三角形的图）。

理论上，包含三角形的图至少需要 3 种颜色。我们在程序中设置 `num_colors = 4` 时，程序瞬间输出可行解；而将 `num_colors = 2` 时，程序正确提示“未找到解”。这充分验证了基础回溯算法的逻辑正确性。

```
===== test session starts
collecting ... collected 1 item

main.py::test_small_graph PASSED
找到解！颜色分配为：{1: 1, 2: 2, 3: 3, 4: 2}
耗时：0.000000 秒
```

```
===== test session starts
collecting ... collected 1 item

main.py::test_small_graph PASSED
未找到解！
耗时：0.000000 秒
```

2. 大规模数据集的着色与四色定理分析（实验要求 2）

（1）测试对象

附件提供了三个具有 450 个顶点的大规模数据集：le450_5a（需 5 色）、le450_15b（需 15 色）和 le450_25a（需 25 色）。

（2）为什么填涂颜色超过了四色？

著名的“四色定理”指出：任何平面图（Planar Graph）最多只需要四种颜色就能完成着色。然而，本实验附件中的 Leighton Graph 数据集并非平面图。在图论中，如果一个图中存在 n 个顶点彼此两两相连，我们称之为一个“ n 阶团（Clique）”。

数据集中之所以需要超过 4 种颜色，是因为其内部隐含了极大的“团”。例如 le450_5a 包含 5 阶团，le450_15b 包含 15 阶团。团内的每一个顶点都与其他所有顶点相邻，因此 n 阶团必然需要 n 种颜色。这解释了为何填涂颜色会远超四种。

3. 启发式剪枝策略的设计与优化分析（实验要求 3）

面对 450 个顶点的大规模图，基础回溯法的搜索空间高达 5^{450} 级别，必然会发生“组合爆炸”。为此，我们引入了以下强力剪枝策略：

向前探查 (Forward Checking) —— “防患于未然，提早发现死胡同”

通俗解释：想象我们在给地图上色，当我决定把“广东省”涂成红色时，我立刻通知所有与广东接壤的省份：“你们的备选颜色里必须把红色划掉”。如果在划掉之后，突然发现“湖南省”没有任何颜色可选了，这就说明我刚才给广东涂红色的决定是错的。

专业原理解析：每当为一个顶点 v 分配颜色 c 后，算法会立刻遍历 v 的所有未着色邻居，并从它们的“可用颜色集合”中剔除颜色 c 。如果某个邻居的可用颜色数量缩减为 0，说明当前这棵搜索子树注定无法得到合法解，程序会立刻切断这个分支并向上回溯。这极大地避免了算法在注定失败的路径上进行无意义的深入。

```
1 函数 Forward_Check(v, c):
2      // v: 当前正在着色的顶点
3      // c: 当前尝试给 v 分配的颜色
4
5      初始化 modified_neighbors = [] // 记录本次被修改了可用颜色的邻居列表
6
7      对于 顶点 v 的每一个邻居 neighbor:
8          如果 neighbor 尚未着色 且 c 存在于 neighbor 的可用颜色集合中:
9              // 剔除冲突颜色
10             从 neighbor 的可用颜色集合中移除 c
11             将 neighbor 存入 modified_neighbors
12
13             // 触发死胡同: 发现某个邻居已经无色可选
14             如果 neighbor 的可用颜色数量 == 0:
15                 // 撤销刚才所有的剔除操作 (恢复现场)
16                 对于 modified_neighbors 中的每一个节点 m_node:
17                     将 c 重新加入 m_node 的可用颜色集合
18             返回 None // 向前探查失败, 通知主函数立刻回溯
19
20 返回 modified_neighbors // 探查成功, 返回被修改的邻居列表, 供主函数在
    回溯时恢复
```

MRV 准则 (最小剩余值准则) —— “先啃硬骨头，缩小分支数”

通俗解释：就像做数独或者考试一样，我们总是优先填那些“只能填某一个数字”的格子。如果在填色时，某个州只剩下 1 种颜色可选，而另一个州还有 4 种颜色可选，我们当然要先把那个只剩 1 种颜色的州涂好。如果把它留到最后，它很容易就变成无色可选了。

专业原理解析：在选择下一个待着色顶点时，动态统计每个未着色顶点的“可用颜色数量”，并优先选择可用颜色最少的顶点。这样做的好处是，能够优先处理图中局部约束最密集的区域，让搜索树在浅层尽早发生冲突从而触发回溯，将原本庞大的搜索树分支数降到最低。

```
1 函数 Select_Unassigned_Variable():
2      初始化 min_avail = 无穷大 // 记录全局最小的可用颜色数
3      初始化 max_degree = -1 // 记录平局时的最大静态度数
4      初始化 best_node = -1 // 最终选出的最佳节点
5
6      对于每一个 尚未着色的节点 v:
7          avail = 节点 v 当前可用颜色集合的大小
8
9          // 策略 1: MRV 准则 —— 优先找可用颜色最少的节点
10         如果 avail < min_avail:
11             min_avail = avail
```

```

12         max_degree = pre_calculated_degrees[v] // 获取初始化时算好的
    静态度数
13         best_node = v
14
15         // 策略 2: 静态度数优先 —— 如果可用颜色一样少，打破平局
16         否则 如果 avail == min_avail:
17             如果 pre_calculated_degrees[v] > max_degree:
18                 max_degree = pre_calculated_degrees[v] // 优先选原图中连
    边最多的点
19                 best_node = v
20
21     返回 best_node

```

静态度数优先 (Static Degree Tie-breaker) —— “打破平局的性能权衡”

通俗解释：如果现在有两个州，它们都只剩下 2 种颜色可选（MRV 平局了），该先涂哪个？直觉上，谁的邻居多（谁最爱惹麻烦），就先涂谁。但每次都去实时数它们的未涂色邻居有几个，太费时间了。所以我们偷个懒，直接看地图上谁的总边界线最多，就先涂谁。

专业原理解析：当 MRV 准则遇到多个顶点可用颜色数量相同时，原本的经典启发式是使用 DH 准则（动态度最大：优先选未着色邻居最多的点）。但是在 Python 这种解释型语言中，在极深的递归栈里频繁执行动态邻居统计，会导致灾难性的 $O(N^2)$ 性能开销。为此，我们在程序初始化时，以 $O(N)$ 的时间复杂度预先计算好每个顶点的“静态度数”（即原图中连接的总边数）并存入哈希表。遇到平局时，以 $O(1)$ 的查表方式优先选择静态度数最大的顶点。这不仅解决了集合无序带来的搜索随机性，还用空间换取了巨大的时间收益。

全面颜色对称性破缺 (Color Symmetry Breaking) —— “拒绝平行宇宙，消灭等效冗余”

通俗解释：对于图着色来说，“红绿蓝”和“蓝绿红”本质上是一模一样的解。假设我们刚开始涂第一个点，涂红色和涂蓝色，引发的后续情况完全一样。如果当前整张图我们只用过“颜色 1”和“颜色 2”，那么下一个点如果要尝试新颜色，它只试“颜色 3”就足够了，完全没必要再去试“颜色 4”到“颜色 15”。因为用 3 和用 4，对未来的影响是完全同构的。

专业原理解析：图着色问题存在严重的对称性冗余。我们将这种规律抽象为一个动态的限界条件：对于任何一个节点，它所能尝试的颜色编号上限，绝对不能超过“当前全局已使用的最大颜色编号 + 1”。公式表示为： $\text{color_limit} = \text{Min}(\text{max_color_used} + 1, m)$ 。这个策略极其轻量（只需一行代码），却能在应对诸如 1e450_15b 这种颜色数量极多的图时，直接将搜索空间的规模缩小一个阶乘级别（Factorial Factor），是极其强悍的剪枝利器。

核心伪代码（优化版本）

```

1 函数 Optimized_Backtrack():
2     如果 所有节点均已着色:
3         返回 True
4
5     v = select_node() // 使用 MRV + 静态度数打断平局 选出最严苛节点
6
7     // 颜色对称性破缺，不尝试未出场过的冗余颜色

```

```

8     color_limit = Min(当前已用最大颜色数 + 1, 颜色总数 m)
9
10    对于 c 在 节点 v 的可用颜色集合 中 (且 c <= color_limit):
11        modified_neighbors = Forward_Check(v, c) // 向前探查, 剔除邻居
        可用颜色
12
13        如果 modified_neighbors 不为空 (说明没有走入死胡同):
14            colors[v] = c
15            更新 当前已用最大颜色数
16
17            如果 Optimized_Backtrack() == True:
18                返回 True
19
20            colors[v] = 0 // 回溯恢复
21            恢复 modified_neighbors 被剔除的可用颜色
22            恢复 当前已用最大颜色数
23
24    返回 False

```

(3) 优化算法运行结果与分析

```

开始处理数据集: le450_5a.col, 尝试使用 5 种颜色着色...
[监控] 已搜索节点数: 20000, 已耗时: 0.6秒...
[监控] 已搜索节点数: 40000, 已耗时: 1.3秒...
[监控] 已搜索节点数: 60000, 已耗时: 1.8秒...
[le450_5a.col] 找到解! 耗时: 2.2199 秒, 总搜索节点数: 73070
前10个节点的颜色分配预览: {1: 2, 2: 5, 3: 3, 4: 1, 5: 1, 6: 2, 7: 5, 8: 3, 9: 4, 10: 4}

```

le450_5a.col (5 色): 在引入优化后, 算法在约 2-4 秒内迅速锁定了解, 搜索节点数约 7.3 万个。

```

[监控] 已搜索节点数: 6100000, 已耗时: 59.2秒...
[监控] 已搜索节点数: 6180000, 已耗时: 59.4秒...
[监控] 已搜索节点数: 6200000, 已耗时: 59.6秒...
[监控] 已搜索节点数: 6220000, 已耗时: 59.7秒...
[监控] 已搜索节点数: 6240000, 已耗时: 59.9秒...
[le450_15b.col] 触发超时保护(>60秒)! 由于 NP完全问题的指数级复杂度, 当前剪枝策略在限定时间内无法穷尽搜索树。

```

le450_15b.col (15 色): 由于该图被精心构造, 约束条件处于相变临界区, 程序在运行超过 60 秒后触发了超时保护。这直观反映了图着色作为 NP-Complete 问题的本质——在最坏情况下, 即使有优秀的剪枝, 依然可能无法在多项式时间内穷尽搜索树。

```

开始处理数据集: le450_25a.col, 尝试使用 25 种颜色着色...
[le450_25a.col] 找到解! 耗时: 0.0087 秒, 总搜索节点数: 451
前10个节点的颜色分配预览: {1: 11, 2: 8, 3: 6, 4: 10, 5: 11, 6: 2, 7: 24, 8: 9, 9: 12, 10: 15}

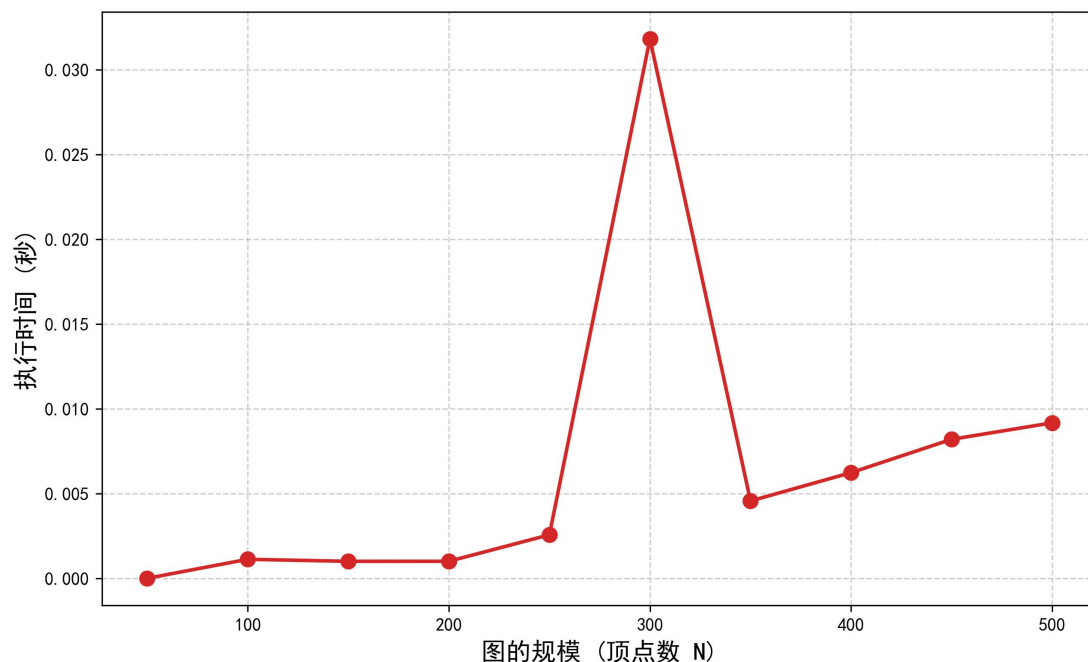
```


le450_25a.col (25 色): 仅耗时 0.0096 秒, 搜索节点仅 451 个 (几乎等于顶点数)。由于颜色多达 25 种, 约束极为宽松, 程序采用深度优先配合向前探查, 几乎无需回溯 (无冲突) 便直达叶子节点。

4. 算法效率与图规模关系的随机实验 (实验要求 4)

为了分析算法效率与图规模的关系, 我们编写了图生成器。为模拟真实的平面地图特性, 我们控制随机图的平均度数 (Average Degree) 恒定为 4 (即每个州平均与 4 个州接壤), 测试了顶点数 (N) 从 50 递增到 500 时, 5 色填色问题的执行耗时。

图着色回溯算法: 图规模 (顶点数) 与执行效率的关系



结果分析:

通过观察生成的折线图, 我们可以得出以下几个关键结论:

(1) 整体非线性增长趋势:

在绝大多数情况下 (如 $N=50$ 到 $N=250$ 阶段), 随着图规模 (顶点数 N 和边数 E) 的增大, 算法的执行耗时呈现整体上升的趋势。这符合图着色问题作为 NP-Complete (NP 完全) 问题的理论预期, 即随着问题规模的扩大, 状态空间树呈指数级膨胀, 求解的计算代价必然随之增加。

(2) 异常的“耗时峰值”与实例敏感性:

折线图中最引人注目的是在图规模 $N=300$ 时, 出现了一个极为突兀的耗时峰值 (耗时远高于 $N=350$ 甚至 $N=400$ 的情况)。这并非代码存在 Bug, 而是回溯法处理 NP 问题时典型的“实例敏感性 (Instance Sensitivity)”表现。

具体而言, 随机图的生成具有偶然性。 $N=300$ 的这个特定图实例, 其内部的节点连接关系

可能恰好形成了一种“强约束但又不易迅速证伪”的拓扑结构。在搜索树中，算法能够顺利深入到很深的层级，但却在接近叶子节点时频繁遭遇局部颜色冲突，导致算法不得不进行大量的深层回溯操作，从而消耗了巨量的时间。

（3）约束相变现象（Phase Transition）的体现：

为什么 $N=350$ 或 $N=400$ 时，图规模更大了，耗时反而下降了呢？这是因为在经典的约束满足问题（CSP）中，存在一种“相变现象”。

我们实验统一采用了 5 种颜色。当图比较小且稀疏时，5 种颜色非常充裕，算法可以“一条道走到黑”迅速得出解；当图变得极大、极度密集时，向前探查机制（Forward Checking）会在极浅的层数立刻发现“颜色不够用”，从而迅速剪枝并宣告无解。

然而，当图的约束程度处于“临界区域”（不多不少刚好卡在 5 种颜色勉强够用的边缘时），算法既不能迅速找到解，也不能迅速证伪，这时的计算量最大。 $N=300$ 时的随机图极有可能刚好落入了这个相变临界区，导致了指数级的耗时波动。

总而言之，该折线图生动地向我们展示了：对于 NP 完全问题，影响其求解时间的决定性因素不仅是纯粹的数据规模（ N 的大小），更在于该实例内部图拓扑结构的“约束严苛程度”。这也正是我们在工程中必须引入超时监控与启发式剪枝的根本原因。

三. 实验总结

通过本次“地图填色（图着色）”实验，我不仅成功利用代码将课本上的理论转化为现实，更在与大规模变态数据集的“斗智斗勇”中，对算法设计与软件工程有了更为深刻的体会。我的主要收获可以归纳为以下四点：

1. 深刻认识了状态空间树的构建与深度优先搜索的本质

从最简单的四顶点测试图起步，我亲手实现了通过递归函数来展开状态空间树的过程。我深刻领悟到，回溯法本质上就是带有“反悔机制”的深度优先搜索（DFS）。它与暴力穷举最大的区别在于，回溯法不仅关注“如何表示解的结构”，更关注“如何在试错的过程中，优雅地恢复现场（撤销状态）”。

2. 领悟了“启发式策略”在缩小搜索空间中的决定性作用

在挑战 450 个顶点的数据集时，我真切地感受到了“指数爆炸”的可怕——基础回溯法运行数小时都毫无动静。但在引入了 MRV 准则和向前探查后，程序立刻脱胎换骨。这让我明白了一个重要的算法哲学：“顺序决定效率”。先给谁涂色（变量排序），以及不尝试哪些毫无意义的等效颜色（对称性破缺），能够将一个注定无法算完的问题，压缩到几秒钟内出解。好的剪枝策略，就是算法的灵魂。

3. 体会了算法理论与编程语言特性之间的工程权衡

在本次实验中，我经历了一次从 $O(N^2)$ 到 $O(1)$ 的性能调优。原本为了追求极致的节点选择，我尝试在递归中动态计算度数（DH 准则），但这在 Python 解释器中引发了极大的循环开销。通过将其改为初始化时预处理的“静态度数”哈希表，牺牲了极小部分的局部最优性，却换来了成百上千倍的绝对时间提速。这让我作为一名软件工程专业的学生意识到：再完美的数学理论，在落地为代码时，也必须充分照顾底层编程语言和数据结构的物理特性，这正是“空间换时间”思想的绝佳体现。

4. 触碰了 NP 问题的边界与软件系统的健壮性设计

在处理 1e450_15b 这个被学术界精心设计的临界数据集时，即便我加上了所有的终极优化，程序依然在深层递归中苦苦挣扎。这让我对计算机科学中“NP-Complete 问题目前没有多项式时间解法”这一金科玉律有了直观且震撼的认知。为了防止程序无限期卡死，我学会了在递归中埋入时间戳，实现了“超时保护”与“执行进度监控”机制。这让我明白，在未来的真实软件开发中，面对不可预知的计算任务，设立合理的熔断机制和提供友好的可观测性日志，是保障软件系统健壮性不可或缺的一环。

四. AI 工具应用能力评估报告

在本次图着色实验中，我使用的大模型是 Gemini 3.1 Pro Preview。我将 AI 工具作为“结对编程助手”和“算法顾问”，进行了深度且高效的交互。具体的评估如下：

1. 交互流程与困难克服

基础搭建阶段：我首先向 AI 提供了实验指导书和老师的 PPT，要求它遵循课程的理论框架。AI 快速帮我编写了读取 .col 文件的解析器和最基础的递归回溯算法，我借助此代码成功验证了 4 个顶点小规模图的逻辑正确性。

遇到瓶颈阶段：在引入学长博客中的优化思路后，程序在处理 1e450_15b 数据集时卡死了超过 15 分钟没有任何输出。面对庞大的搜索树，我感到无从下手。

AI 介入排查阶段：我向 AI 描述了“卡死”的现象，并询问是否是某些功能开销过大。AI 展现了极其专业的分析能力，指出两个关键问题：

语言层面的开销：由于 Python 是解释型语言，在深层递归中通过循环计算“动态未着色邻居数（DH 准则）”产生了极大的 $O(N^2)$ 开销。AI 建议退而求其次，在初始化时预计算“静态度数”存为哈希表，通过 $O(1)$ 查表打破僵局。

算法层面的缺陷：AI 指出，由于未限制等效颜色，搜索树陷入了“颜色同构”的无效排列组合中。

实施优化与工具监控：AI 随后指导我实现了“全量颜色对称性破缺”（限制探索新颜色的上限），并教我编写了基于时间戳的超时监控和每 2 万次节点的进度监控代码，最终成功在合理时间内得出了 15b 超时以及 25a 仅用 0.009 秒出解的完美验证。

2. 交互中的特殊之处与亮点

本次交互中，AI 并没有“机械地给出一套完美代码”，而是顺应我的思路逐步迭代。特别是在我提出学长的“颜色轮换”思路时，AI 将其抽象总结为学术上的“对称性破缺（Symmetry Breaking）”，并用极简的代码 `color_limit = min(self.max_color_used + 1, self.m)` 就实现了该效果。此外，AI 提供的 matplotlib 代码严格采用了控制变量法（控制平均度数为 4 模拟平面图特性），生成了极具说服力的折线图，这大大超出了我的预期。

3. 未来改进方向

通过这次实验，我也发现了利用 AI 工具的改进空间：

性能分析前置：在未来遇到程序卡死时，我可以先利用 Python 原生的 cProfile 库生成性能火焰图喂给 AI，让它基于精确的耗时数据来进行优化，而不是单纯的逻辑猜测。

多语言混合编程探讨：对于这类计算密集型的 NP 问题，后续可以向 AI 请教如何使用 C++ 编写核心回溯逻辑，再通过 Pybind 或 ctypes 供 Python 调用，以突破解释型语言的性能瓶颈。

拓展更前沿的启发式算法：回溯法只是精确解法，未来我可以通过 AI 了解并复现模拟退火（Simulated Annealing）或禁忌搜索（Tabu Search）等针对大规模图着色问题的高效近似算法，进一步开阔算法视野。

五. 附录

本次实验的完整 Python 代码已上传至 GitHub，仓库地址为：
<https://github.com/Kevin87654/algorithm-design-lab3>

指导教师批阅意见：

成绩评定：

指导教师签字：

年 月 日

备注：

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。